

Minimum Window Subsequence Approach

Minimum Window Subsequence — Approach + Intuition + Code

The important word here is **subsequence**, not substring.

For example:

```
s1 = "abcdebddde"
s2 = "bde"
```

Inside "bcde":

```
b c d e
↑  ↑ ↑
```

We can skip characters (c), but the characters of s2 must appear **in the same order**.

We need the **smallest substring of s1** that contains s2 as a subsequence.

1. Basic intuition

Take:

```
s1 = abcdebddde
s2 = bde
```

We scan s1 looking for:

```
b → d → e
```

The first time we successfully find all characters:

```
abcde
↑  ↑ ↑
b  de
```

Now we know:

```
"abcde"
```

is a valid window.

But is it the smallest?

No, because the window starts unnecessarily early.

So once we find a complete subsequence, we should try to **shrink the window from the left**.

2. Forward scan + backward shrink

This is the easiest and most intuitive approach.

Step A: Move forward

Find the complete sequence:

```
s1 = a b c d e b d d e
      ↑   ↑ ↑
      b   d e
```

Suppose we finish matching s2.

Now we have a valid window:

```
[a b c d e]
```

But it may contain unnecessary characters at the beginning.

Step B: Move backward

Starting from the **e** we found, move backward and match s2 in reverse:

```
s2 = b d e
      ↑
```

We already found e.

Now search backward for d:

```
a b c d e
      ↑
```

Then search backward for b:

```
a b c d e
      ↑
```

So the actual minimum window is:

```
b c d e
```

Therefore:

```
"bcde"
```

3. Complete working

We use two pointers:

```
i -> traverses s1
j -> traverses s2
```

Forward matching

```
C++
if (s1[i] == s2[j])
    j++;
```

When:

```
j == s2.size()
```

it means we found the complete subsequence.

For example:

```
s1 = abcdebdde
s2 = bde

Forward matching:
a b c d e
  ↑   ↑   ↑
  b   d   e

j == 3
```

Now we have a valid window ending at i.

Backward shrinking

Now reset:

```
C++
j = s2.size() - 1;
```

and move backward.

```
s1 window = abcde
s2         = bde
```

Find e:

```
a b c d e
      ↑
```

Then d:

```
a b c d e
  ↑
```

Then b:

```
a b c d e
  ↑
```

Now we know the smallest possible starting position for this ending position.

So:

```
start = position of b
end   = position of e
```

Window:

```
bcde
```

4. Visual dry run

```
s1 = a b c d e b d d e
    0 1 2 3 4 5 6 7 8
s2 = b d e
```

Forward scan

```
i = 0
a != b
```

Continue:

```
i = 1
s1[i] = b
s2[j] = b
MATCH
j++
```

```
Matched: b
```

Continue:

```
c -> ignore
```

Then:

```
i = 3
s1[i] = d
s2[j] = d
MATCH
```

Then:

```
i = 4
s1[i] = e
s2[j] = e
MATCH
```

Now:

```
j == s2.length()
```

We found:

```
abcde
```

But now shrink it.

Backward scan

Start from:

```
a b c d e
      ↑
      e
```

Match:

```
e
```

Move backward.

Find:

```
a b c d e
      ↑
      d
```

Move backward.

Find:

```
a b c d e
      ↑
      b
```

Therefore:

```
start = 1
end = 4

window = "bcde"
```

Store it as the current answer.

Then continue searching after this window.

5. C++ Code

```
C++
class Solution {
public:
    string minWindow(string &s1, string &s2) {
        int n = s1.size();
        int m = s2.size();

        int minLen = INT_MAX;
        int startIndex = -1;

        int i = 0;
        while (i < n) {
            int j = 0;
            // Forward scan: find s2 as a subsequence
            while (i < n) {
                if (s1[i] == s2[j]) {
                    j++;
                }

                if (j == m) {
                    break;
                }

                i++;
            }

            // Could not find complete s2
            if (j != m) {
                break;
            }

            // Current valid window ends at i
            int end = i;

            // Backward scan: minimize the window
            j = m - 1;
            while (j >= 0) {
                if (s1[i] == s2[j]) {
                    j--;
                }

                i--;
            }
        }
    }
};
```

```
// i went one position before the actual start
int start = i + 1;

int len = end - start + 1;

// Update only if strictly smaller
// This automatically keeps the leftmost window
// when lengths are equal.
if (len < minLen) {
    minLen = len;
    startIndex = start;
}

// Continue searching after the start
i = start + 1;
}

if (startIndex == -1) {
    return "";
}

return s1.substr(startIndex, minLen);
}
};
```

6. Let's understand the tricky part

The most confusing part is usually this:

```
C++
j = m - 1;

while (j >= 0) {
    if (s1[i] == s2[j]) {
        j--;
    }
    i--;
}

int start = i + 1;
```

Suppose:

```
s1 = a b c d e
      indexes
      0 1 2 3 4
s2 = b d e
```

Initially:

```
i = 4
j = 2
```

Because:

```
s1[4] = e
s2[2] = e
```

Match:

```
j = 1
i = 3
```

Now:

```
s1[3] = d
s2[1] = d
```

Match:

```
j = 0
i = 2
```

Now move backward:

```
s1[2] = c
s2[0] = b
```

No match:

```
i = 1
```

Now:

```
s1[1] = b
s2[0] = b
```

Match:

```
j = -1
i--
```

So now:

```
i = 0
```

But the actual start was:

```
1
```

Therefore:

```
C++
int start = i + 1;
```

7. Why do we write this?

```
C++
i = start + 1;
```

After finding one minimum window:

```
bcde
↑
start
```

We need to continue searching for another possible window.

For example:

```
abcdebdde
  └───┘
    bcde
      └───┘
        b d d e
```

So we continue from:

```
start + 1
```

This ensures we don't miss overlapping windows.

8. Example with the tie case

```
s1 = abcdebdde
s2 = bde
```

Possible windows:

```
bcde
bdde
```

Both have length:

```
4
```

The answer should be:

```
bcde
```

because it appears first.

That is why we use:

```
C++
if (len < minLen)
```

and **not**:

```
C++
if (len <= minLen)
```

With <:

```
First minimum window -> stored
Second equal-size window -> ignored
```

So the leftmost answer remains.

9. Time Complexity

Let:

```
n = s1.length()
m = s2.length()
```

The forward and backward scans can revisit characters while searching for windows.

Worst case:

```
O(n × m)
```

For the constraints:

```
s1.length <= 10^4
s2.length <= 50
```

this approach is efficient enough.

Space:

```
O(1)
```

We only use pointers and variables.

The main idea to remember

1. Scan forward until s2 is completely matched.

↓
Valid window found
2. From that ending position,
 scan backward and match s2 in reverse.

↓
Minimum start found
3. Update answer.
4. Start searching again.

In one line:

Find a valid window by moving forward, then minimize that same window by moving backward.

This is the key pattern for **Minimum Window Subsequence**.

ChatGPT can make mistakes. Check important info.